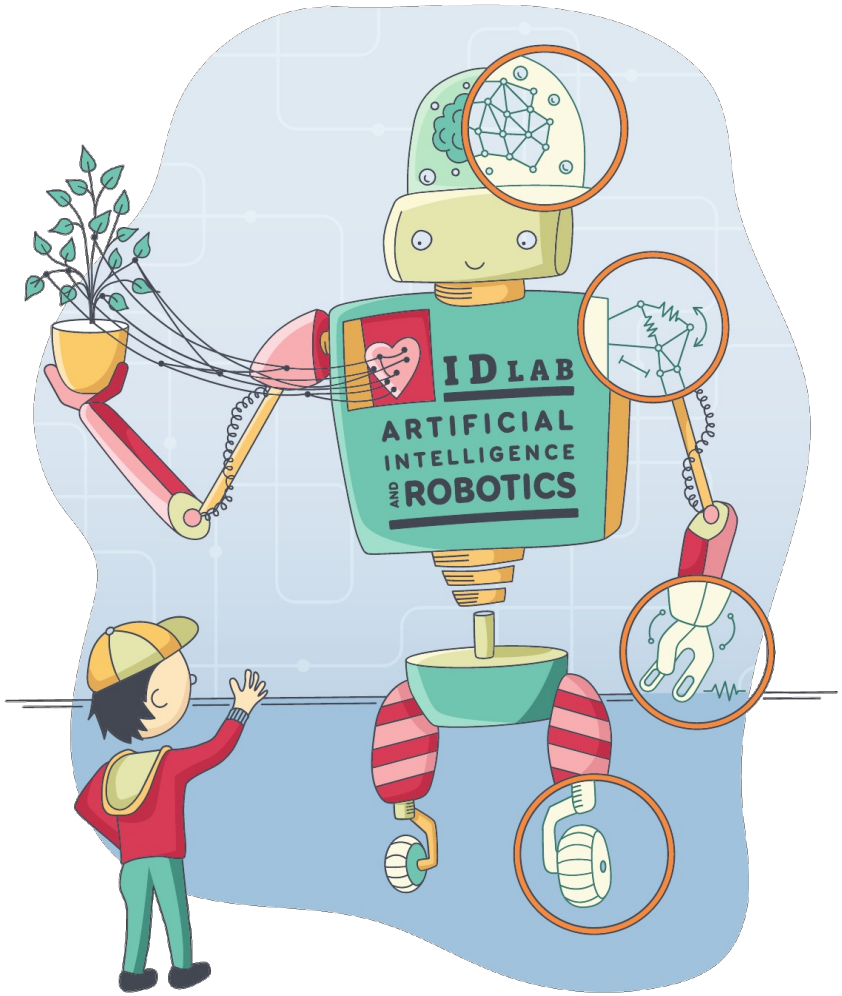
Planning for robotic manipulation

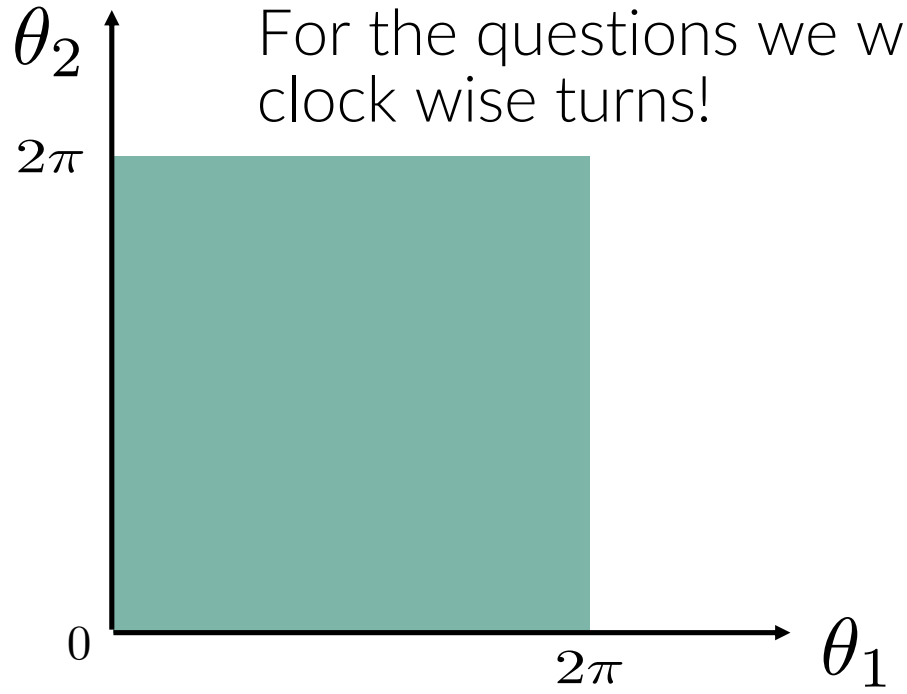


GHENT UNIVERSITY imec IDLab INTERNET & DATA LAB

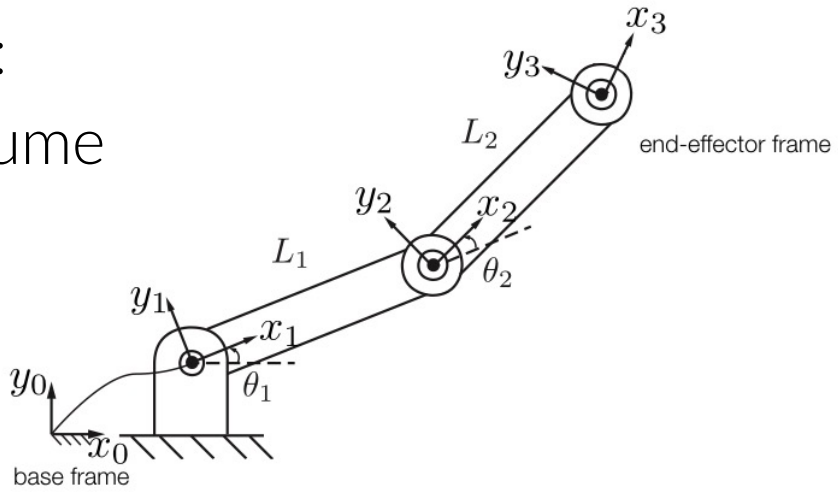
Quiz

Given a planar robot with two revolute joints θ_1, θ_2 in $[0, 2\pi]$

The configuration space will look like this:



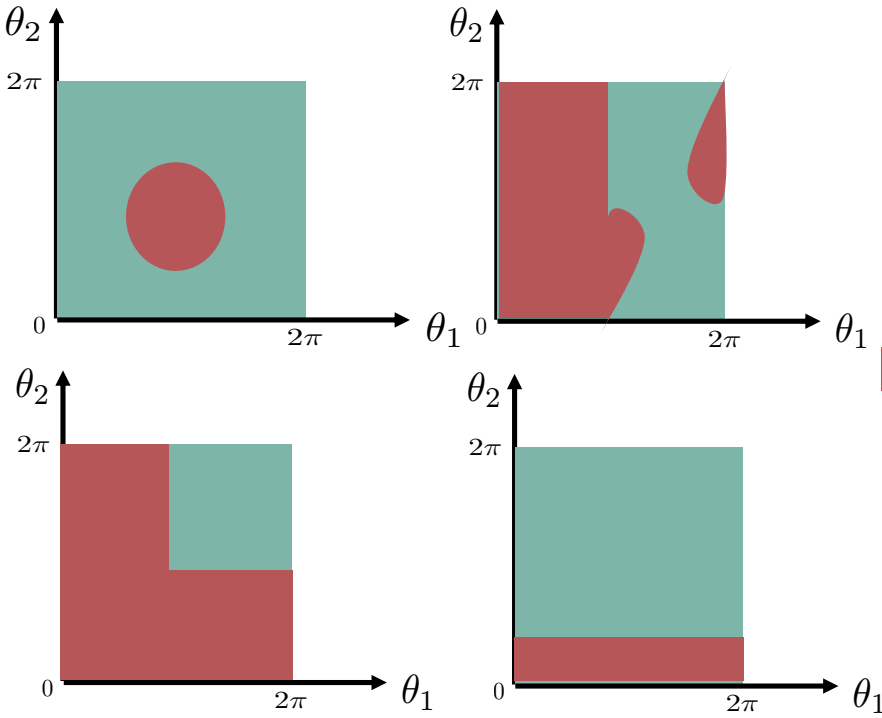
For the questions we will assume clock wise turns!

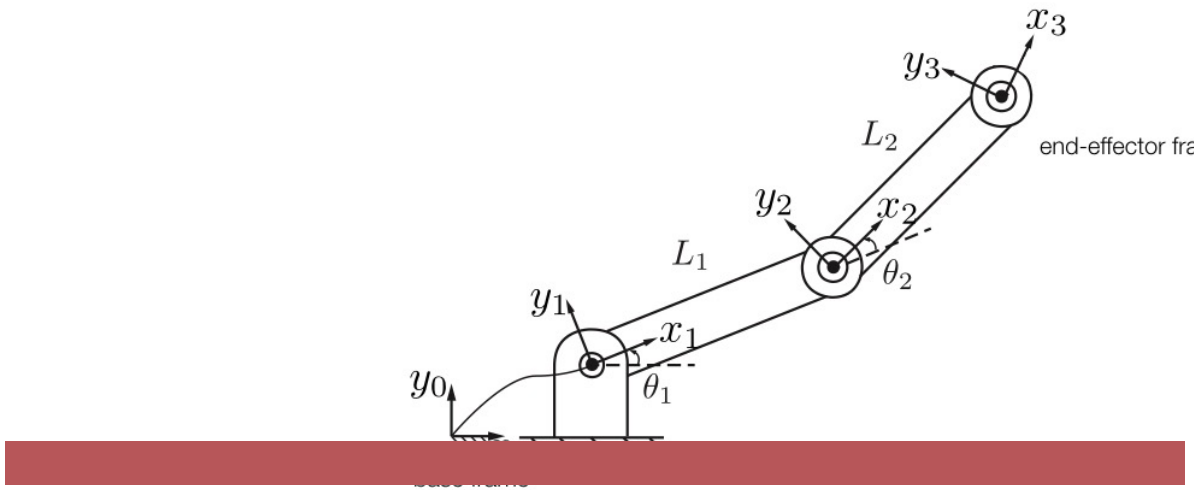


$$x3 = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$
$$y3 = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

Quiz

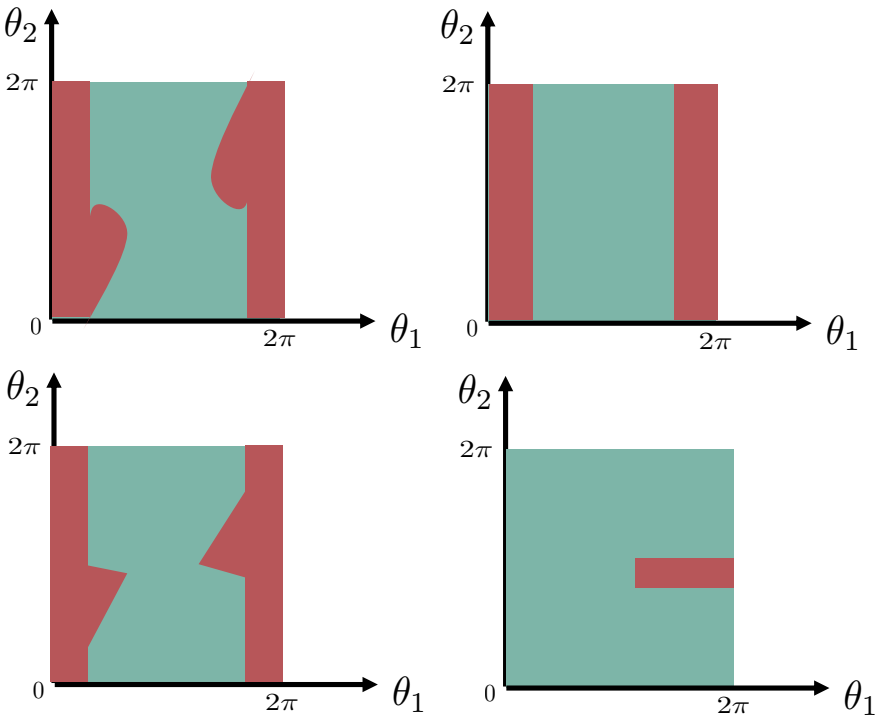
How will the configuration space look when we put the red obstacle?

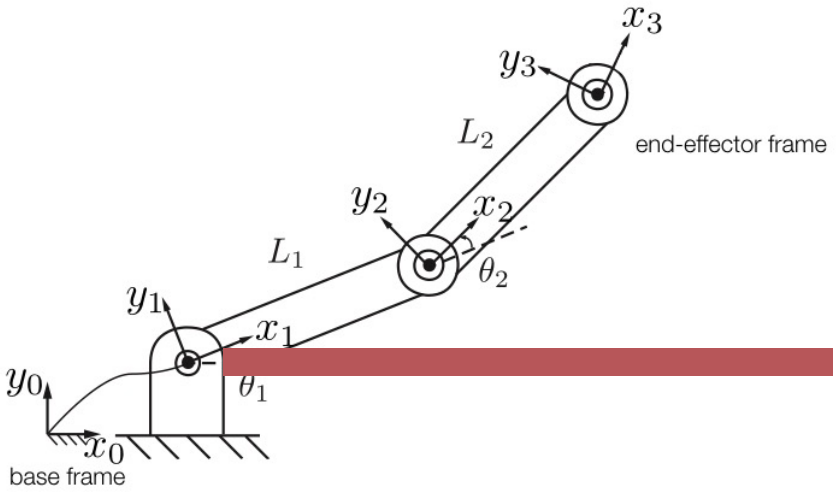




Quiz

How will the configuration space look when we put the red obstacle?



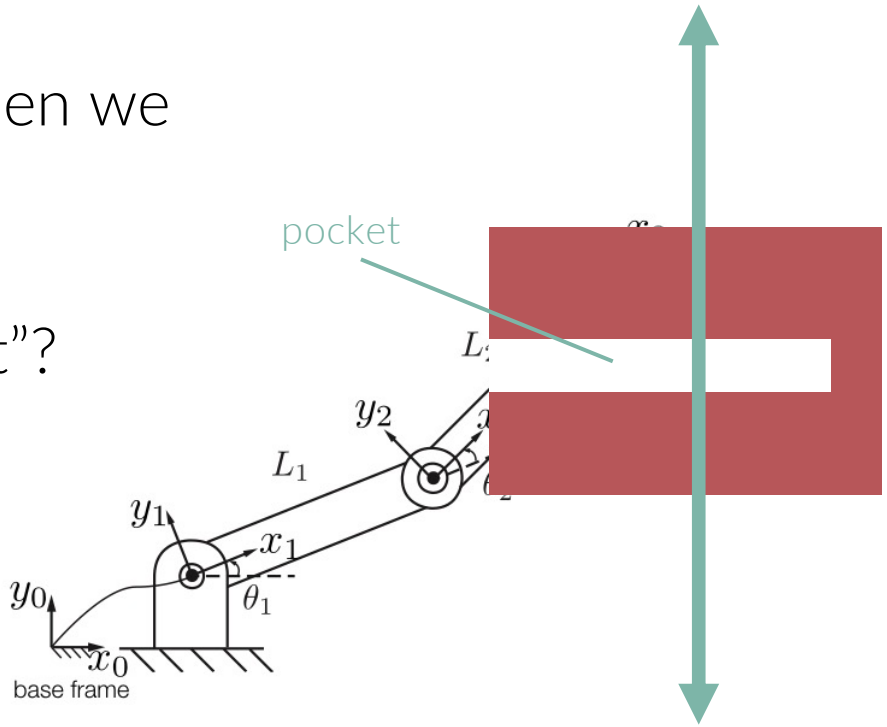


Quiz

How will the configuration space look when we put the red obstacle?

How deep can the TCP reach the “pocket”?

Let’s have a look …

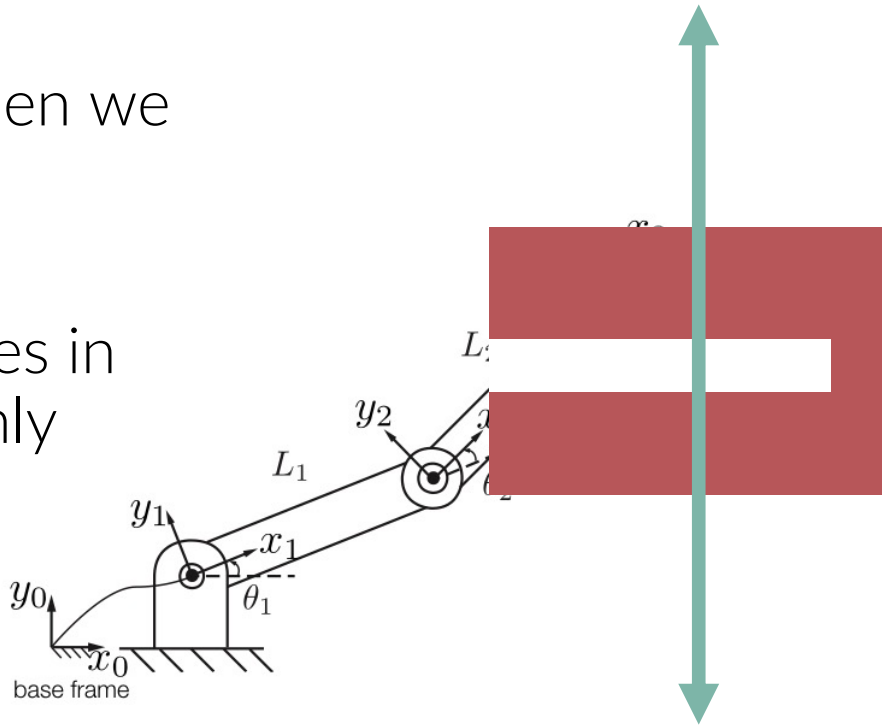


Quiz

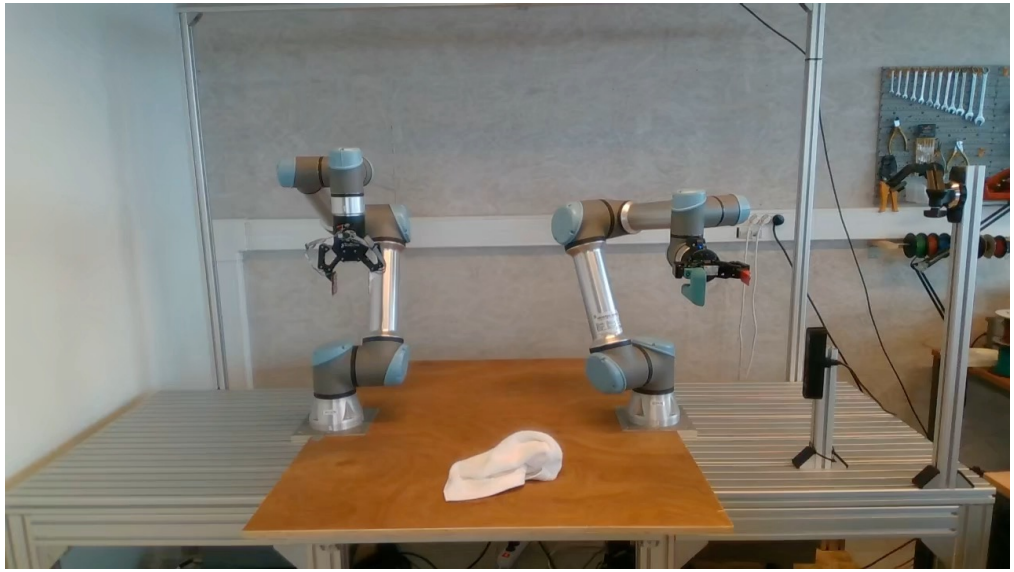
How will the configuration space look when we put the red obstacle?

It's hard to interpret the effect of obstacles in configuration (joint) space due to the highly non-linear transformations

-> Planning is more complex then you would think on first sight



Planning isn't easy: avoid obstacles and self-collisions



Today's topics

- Planning terminology
  - Path and motion planning
  - Path vs. Trajectory
  - Configuration space
  - Planning methods
- Grid-based planning
- Sampling-based planning
- Optimization-based planning

Planning terminology

The state of a robot

The state of a robot is defined by its configuration and joint velocities:

$$x = (q, \dot{q})$$

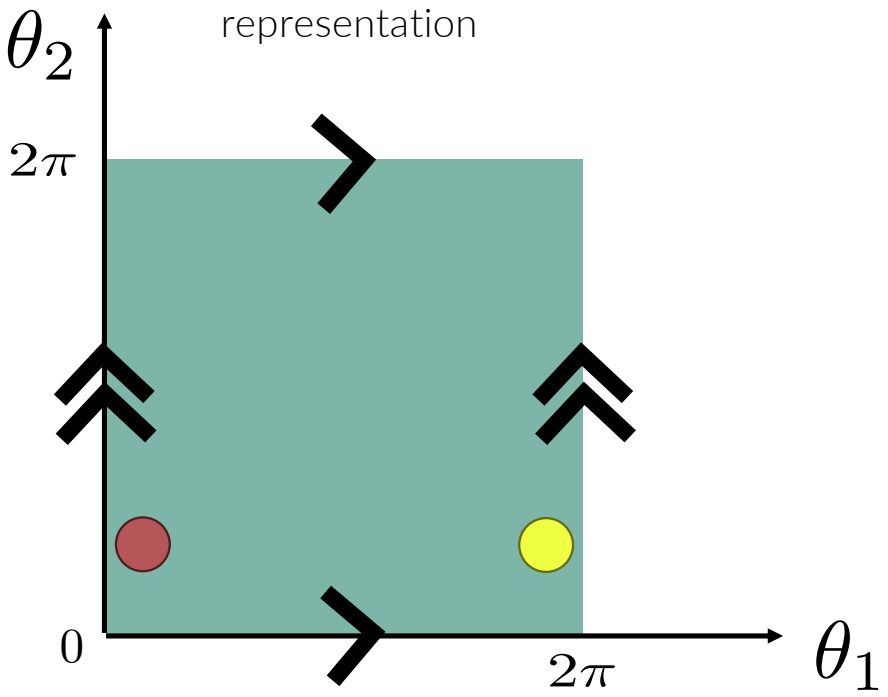
The configuration space

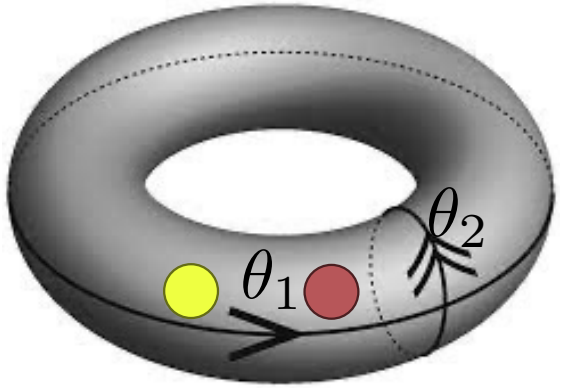
The configuration space (C-space) is defined by all possible configurations

$$q = (q_1, q_2, ..., q_N)$$

with N the degree of freedom

Note: there is also the work space = the configurations that the end-effector (TCP) can reach





topology

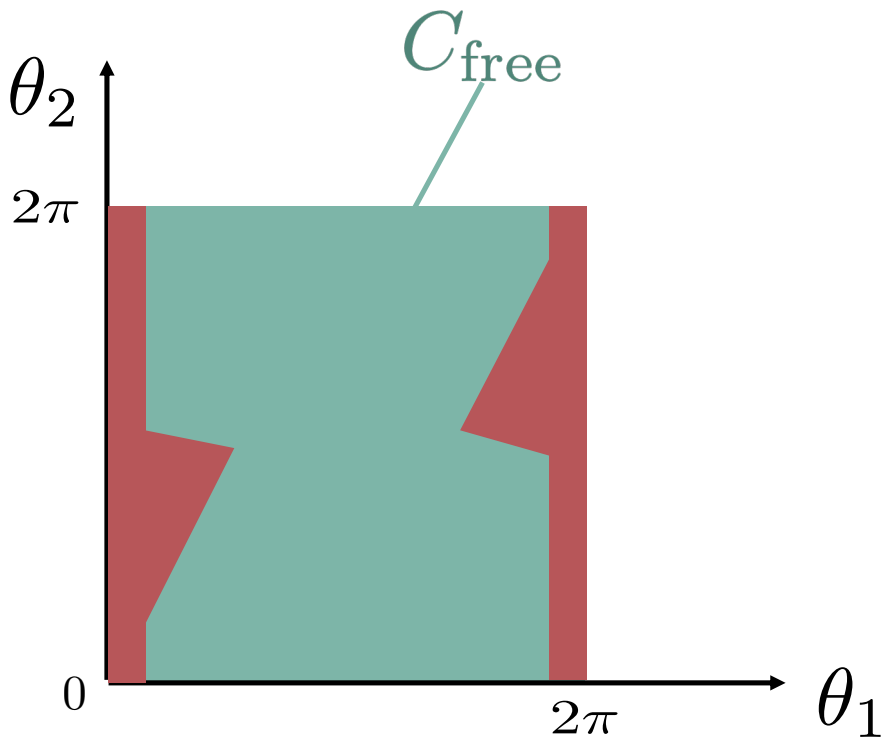
The free configuration space

The configuration space (C-space) is defined by all possible configurations

$$q = (q_1, q_2, ..., q_N)$$

with N the degree of freedom

The subspace of C where the robot can go without collision



Motion planning: a definition

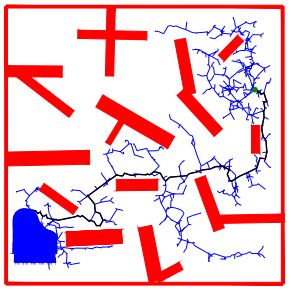
Given the robot's state $x = (q, \dot{q})$ and initial state $x(0) = x_{start}$ and a desired final state x_{goal}

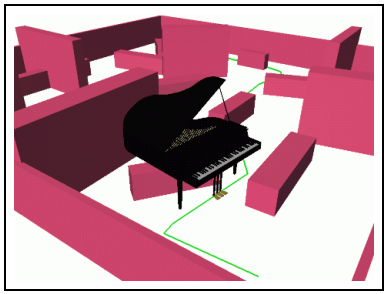
Then motion planning can be seen as finding a time T and a sets of robot states $x(t)$ with t in $[0,T]$ that satisifies $x(T) = x_{goal}$ and $q(x(t))$ is in the C_{free} on every time step

Trajectory: $x(t)$ for t in $[0,T]$

Path planning

- Path planning can be seen as a subproblem of the motion planning problem
- A pure geometric problem
- Deals with finding a collision free path $q(s)$ with s in $[0,1]$ from a start configuration $q(0) = q_{start}$ to a goal configuration $q(1)=q_{goal}$ without concern for the dynamics, the duration of motion or constraints on the motion or on the control inputs.





Moving a piano – it's ok to take time as long it fits

Planning in configuration or work space

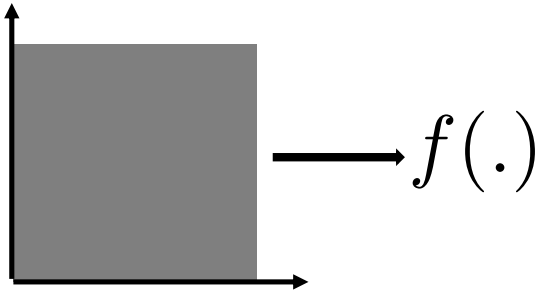
In configuration space

- Used more often in the context of robotic manipulation
- More precise control of the trajectories (e.g., smoothness)
- Exploit kinematic redundancies
- Collision checking is trivial if you have them in C-space
- C-space can become very large for very articulated robots (many DOFs)
- Wandering

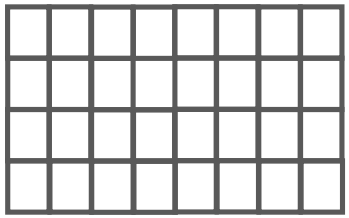
In work space

- Easier to interpret
- Easier when thinking about specific tasks (e.g., moving a glass of water)
- No wandering
- Requires computing of the inverse kinematics the check collisions with the robot
- Harder to get smooth trajectories
- Small TCP movement can require large joint movement

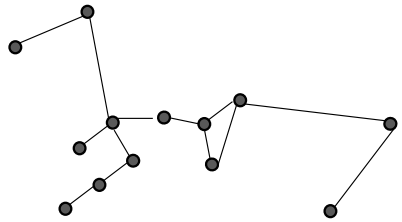
Planning methods



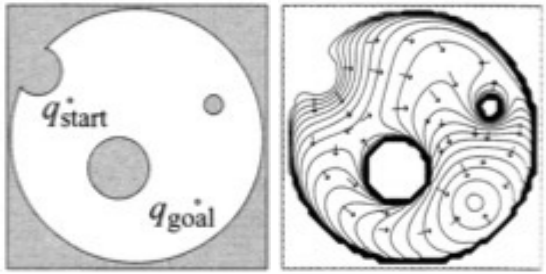
Complete methods



Grid-based methods



Sampling-based methods

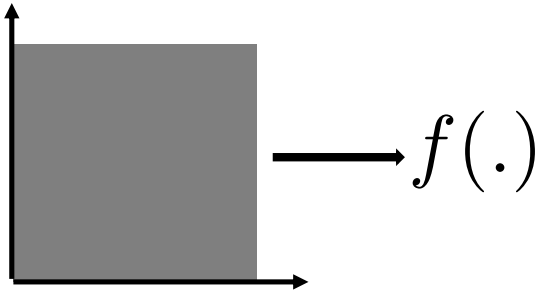


Virtual potential fields

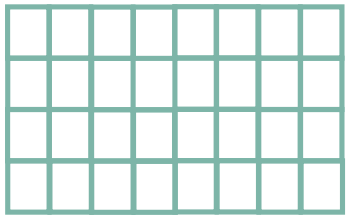
$$\min_{v} \left|\left| J(q)v - V \right|\right|^2$$

Non-linear optimalisation

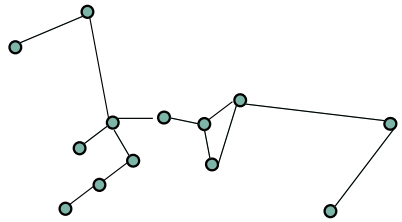
Planning methods



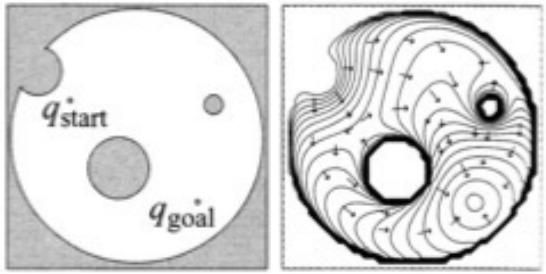
Complete methods



Grid-based methods



Sampling-based methods



Virtual potential fields

$$\min_{v} \left|\left| J(q)v - V \right|\right|^2$$

Non-linear optimalisation

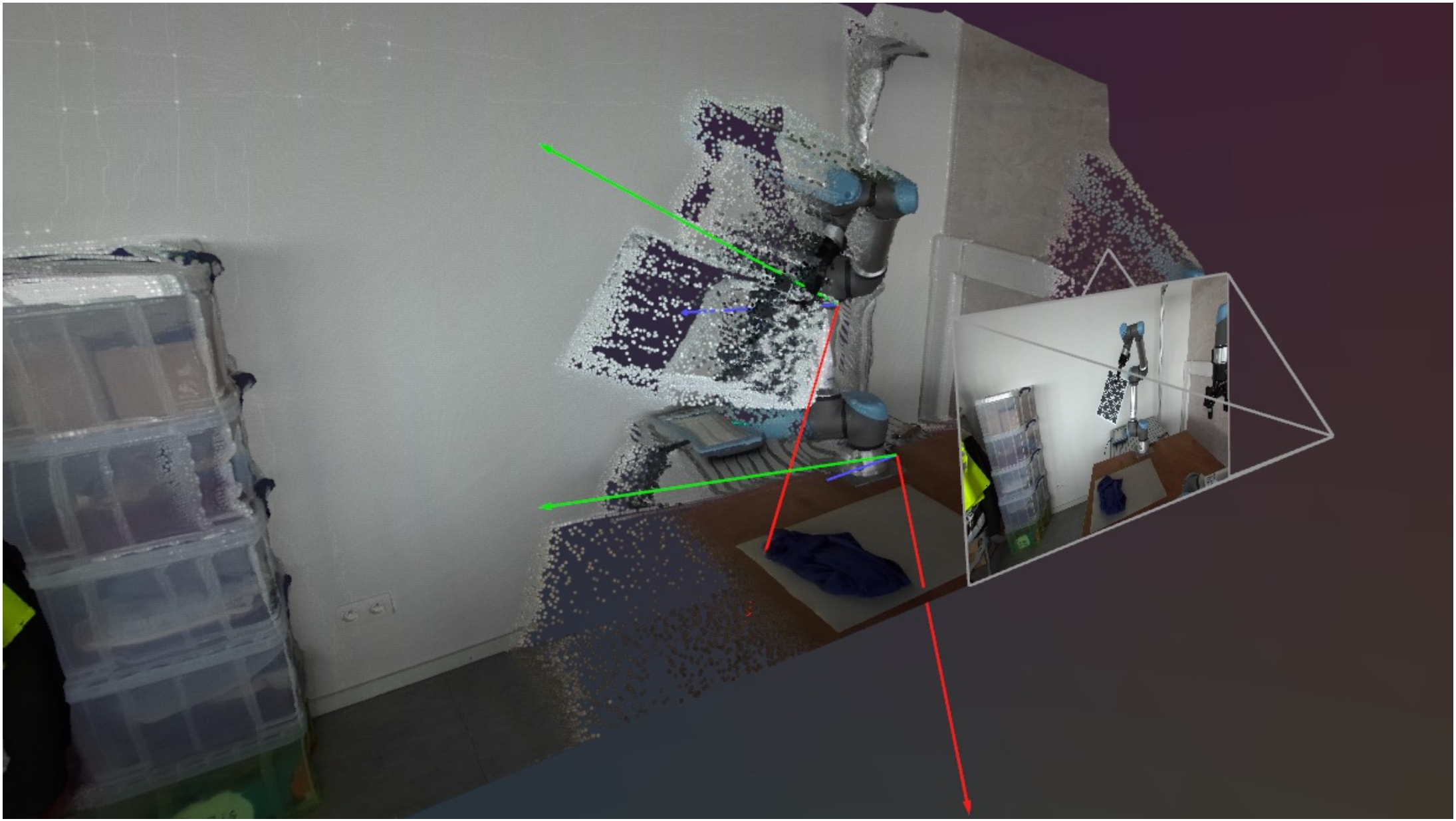
What do we want from planners?

- Complete: guaranteed to find a solution (if it exists) or not?
- Computational efficient

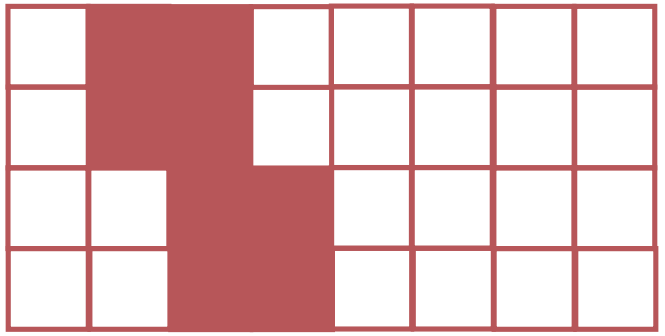
Additional properties:

- Multiple-query or single-query: starting from scratch or not
- Anytime planner: continues, even after finding a first (possibly sub-optimal) solution
- Online versus offline
- Optimal versus satisficing
- Exact versus approximate

Grid methods



Space partitioning technique occupancy grid maps



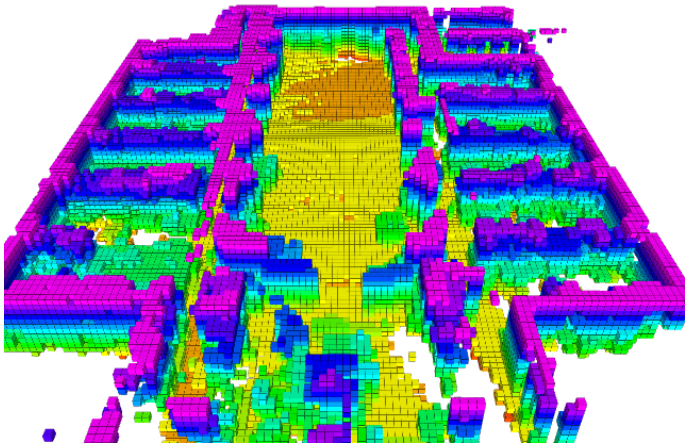
- Models the occupancy of the space

  -> each cell is either occupied or free (binary value)

- Discretize the world into cells

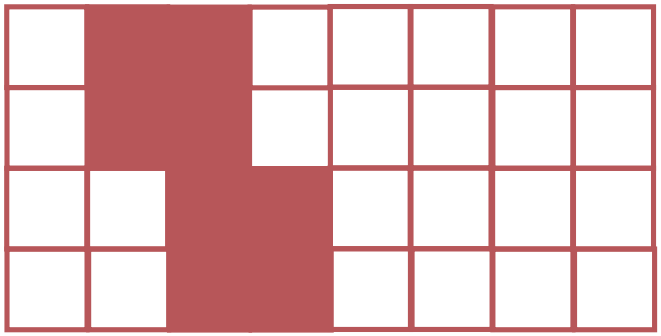
  -> Curse of dimensionality!

- Typically one assumes a static environment
- Can be represented by a graph

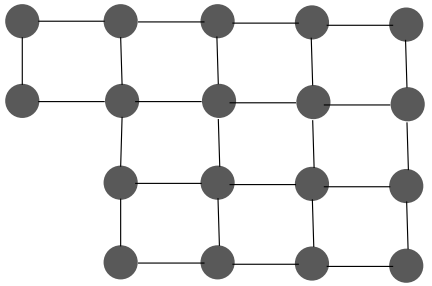


GHENT UNIVERSITY imec IDLab INTERNET & DATA LAB

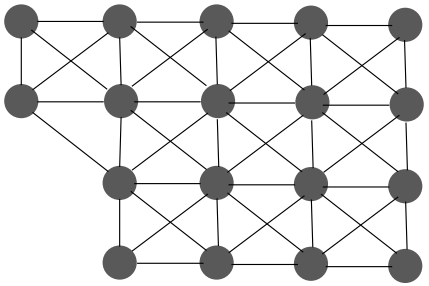
Space partitioning techniques: occupancy grid maps



Not the most efficient way of representing a free space
-> octree

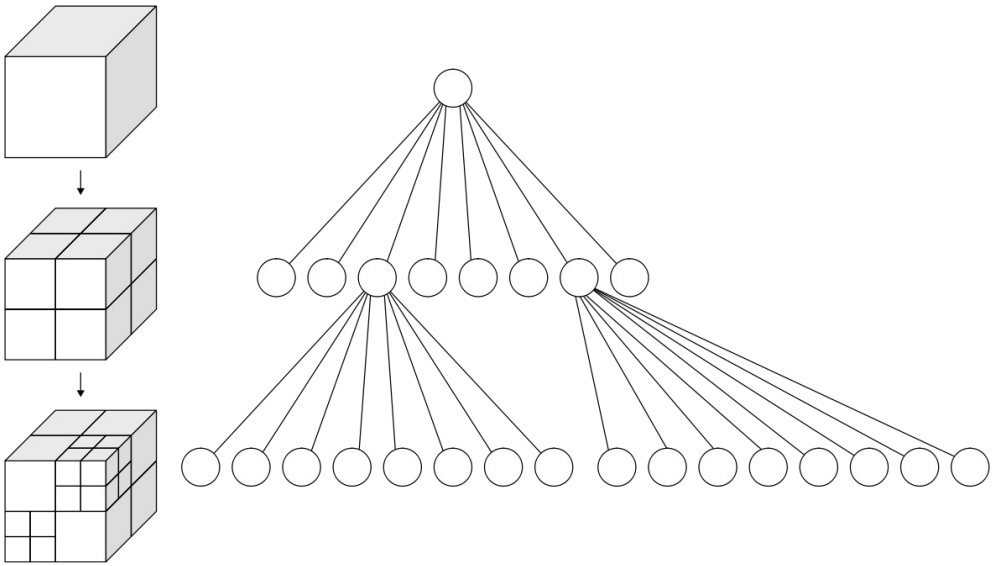


4-neighbours method



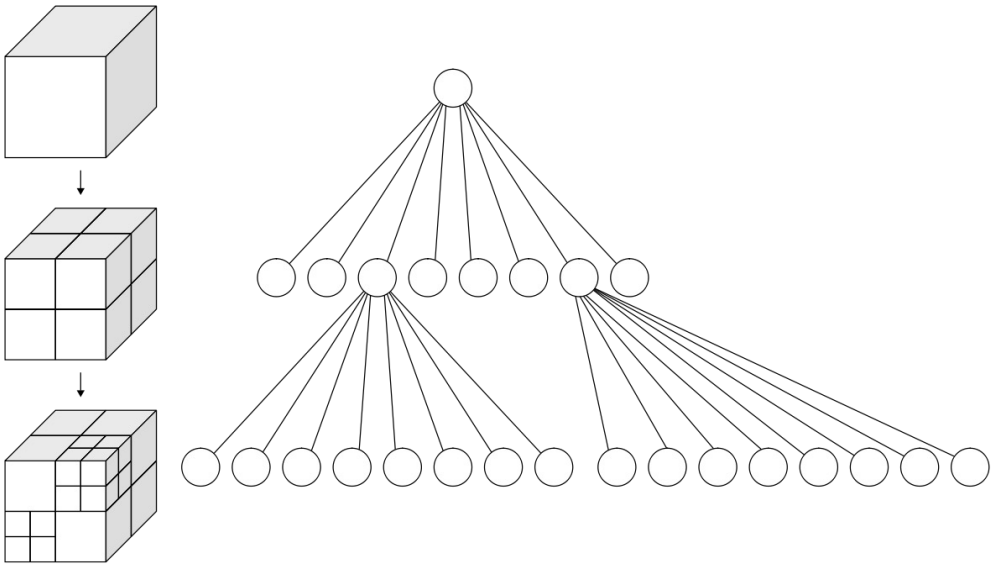
8-neighbours method

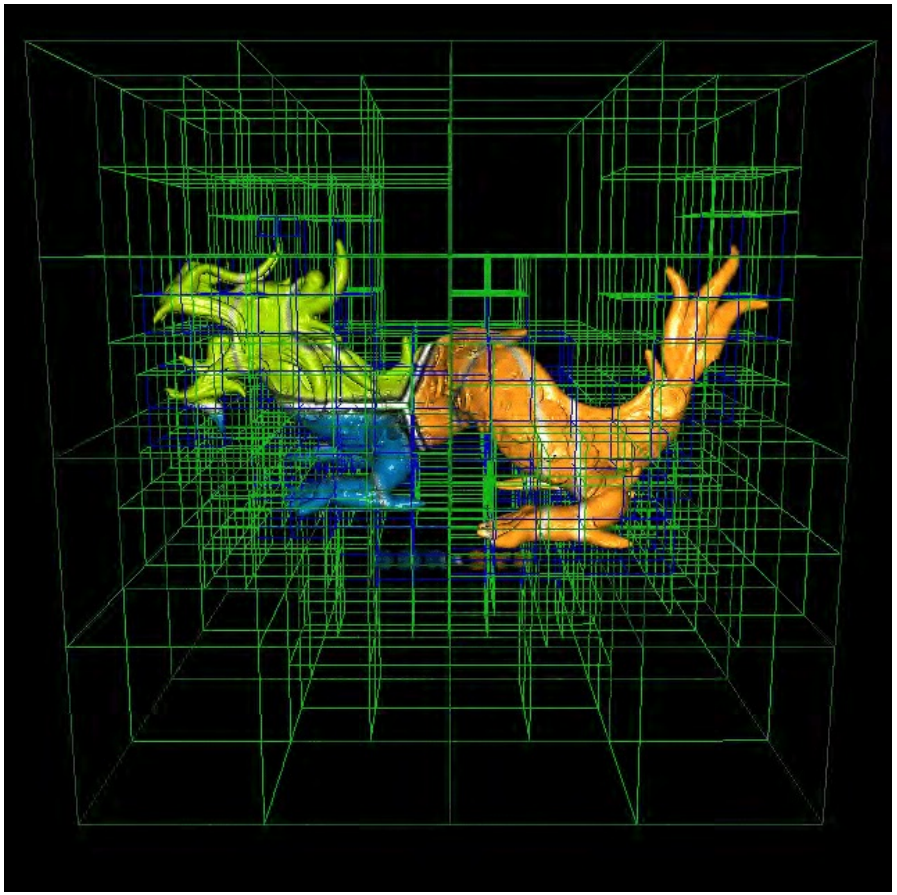
Space partitioning techniques: occupancy grid maps with Octrees



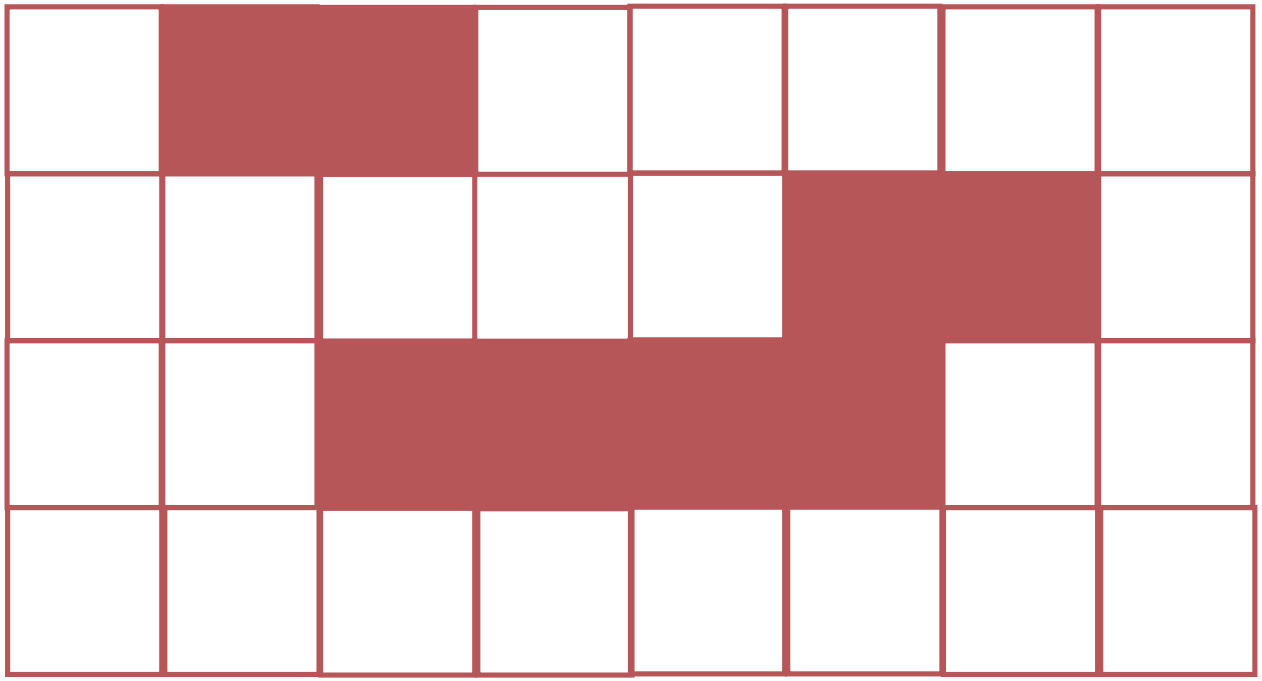
- Representation for the free space: a node represents a free volume
- Each node has at most 8 children
- If you encounter an obstacle -> divide further

Space partitioning techniques: occupancy grid maps with Octrees





Planning as a shortest path problem: A*

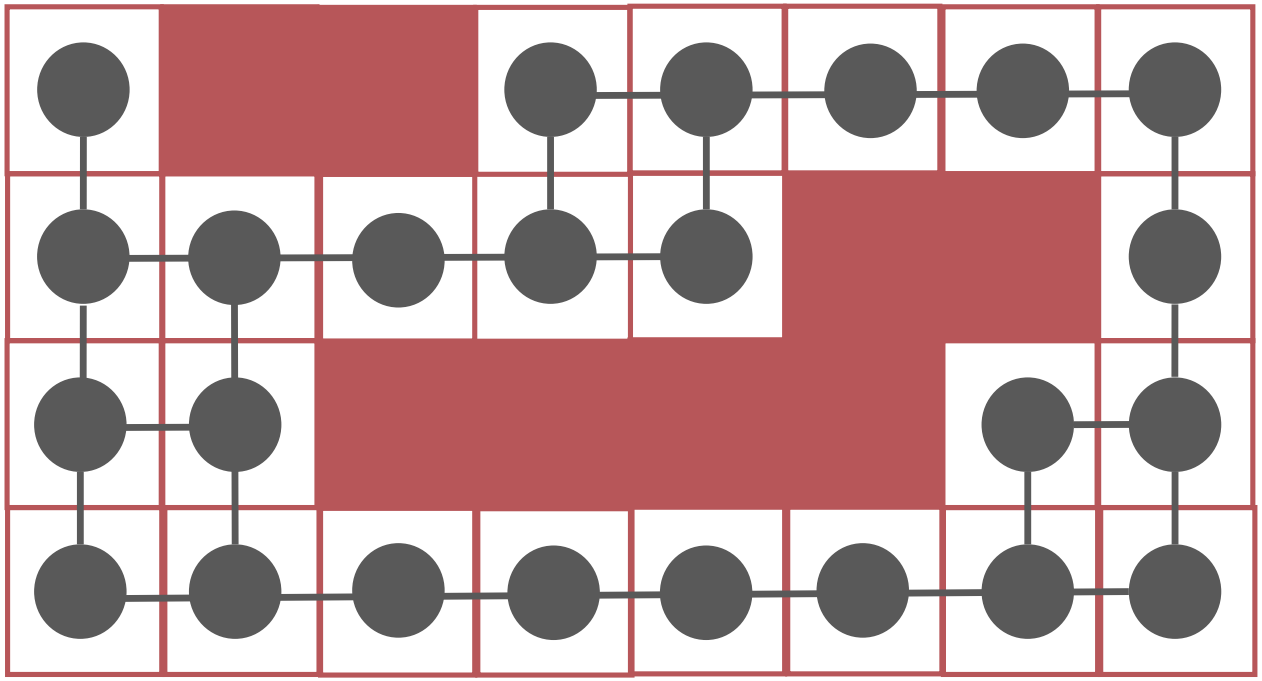


Planning as a shortest path problem: A*

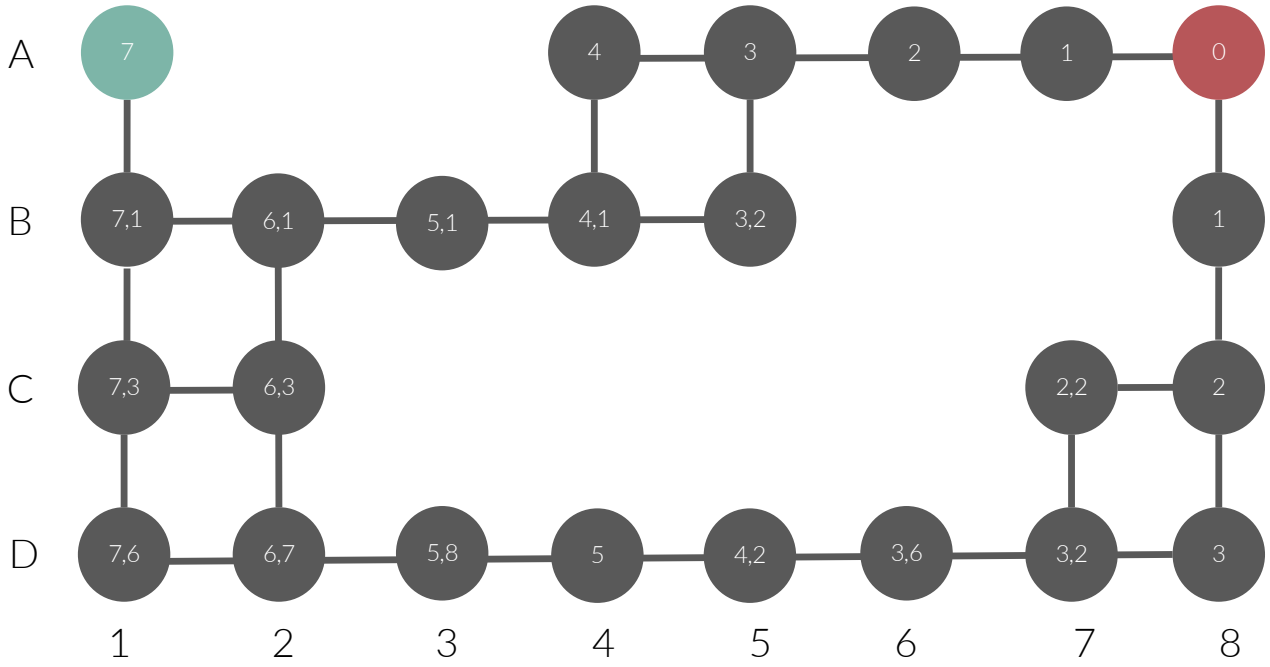
```
make an open list containing only the starting node
make an empty closed list


while (not at the goal configuration):
         consider the node with the lowest F score in the open list
         if (this node is our goal):
                  finished
         else: put the current node in the closed list and look at all neighbours
                  for (each neighbour):
                     if (neighbour has lower G value than current and is in the closed list):
                           replace the neighbour with the new lower G value
                           current node is now the neighbour's parent
                     else if (current G value is lower and this neighbour is in the open list):
                           replace the neighbour with the new lower G value
                           change the neighbour's parent to our current node
                     else if this neighbour is not in both lists:
                           add it to the open list and set its G
```

Planning as a shortest path problem: A*



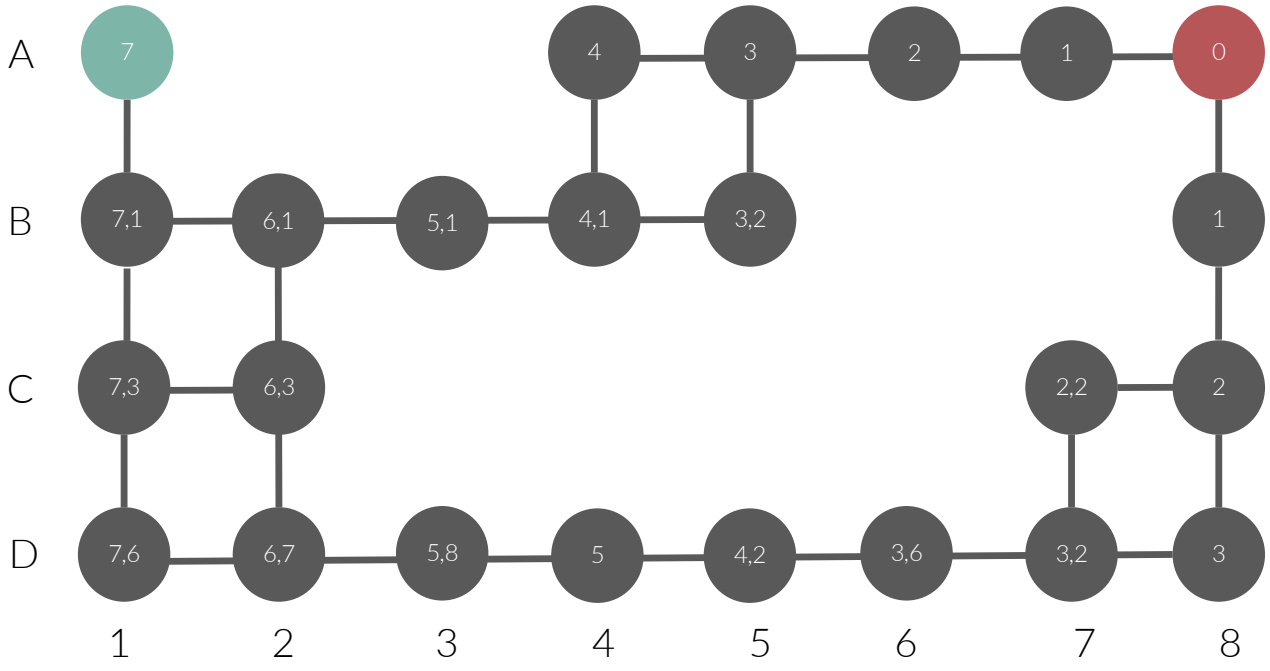
Planning as a shortest path problem: A*



weight = 1

We also calculated how far (in this case euclidian distance) a node is from the goal node

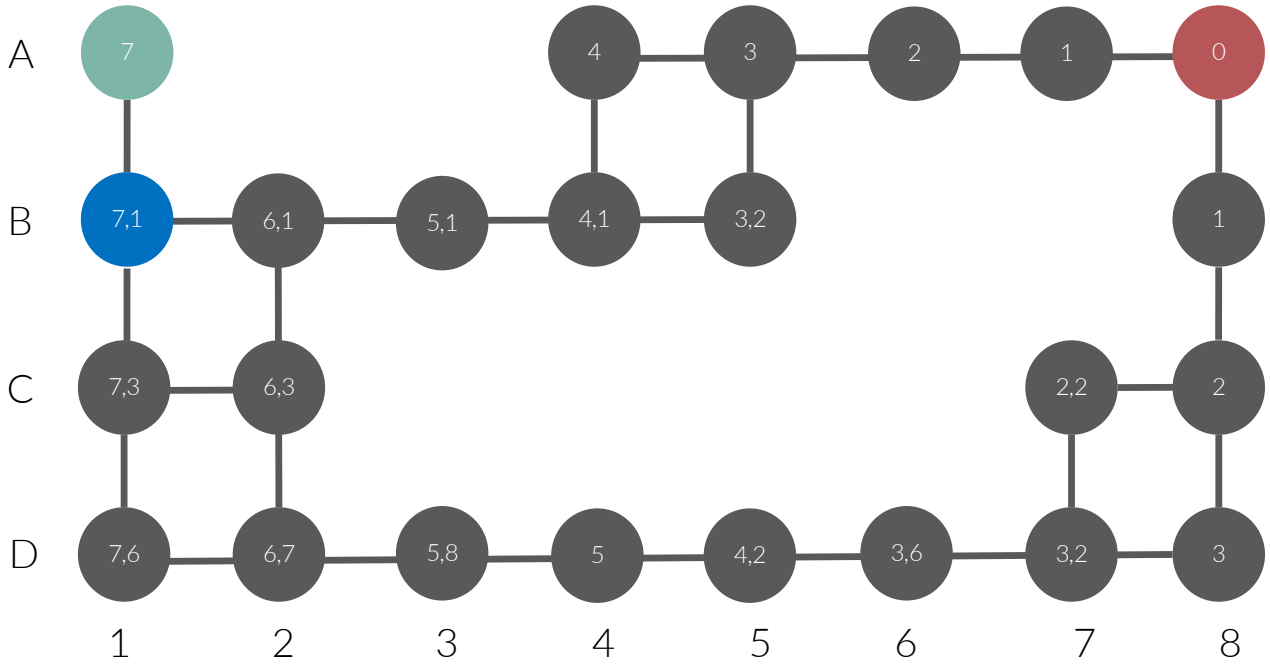
Planning as a shortest path problem: A*



Node	Value	Parent
A1	7+0=7	none

Value (F) = heuristic cost (H) + walking cost (G)

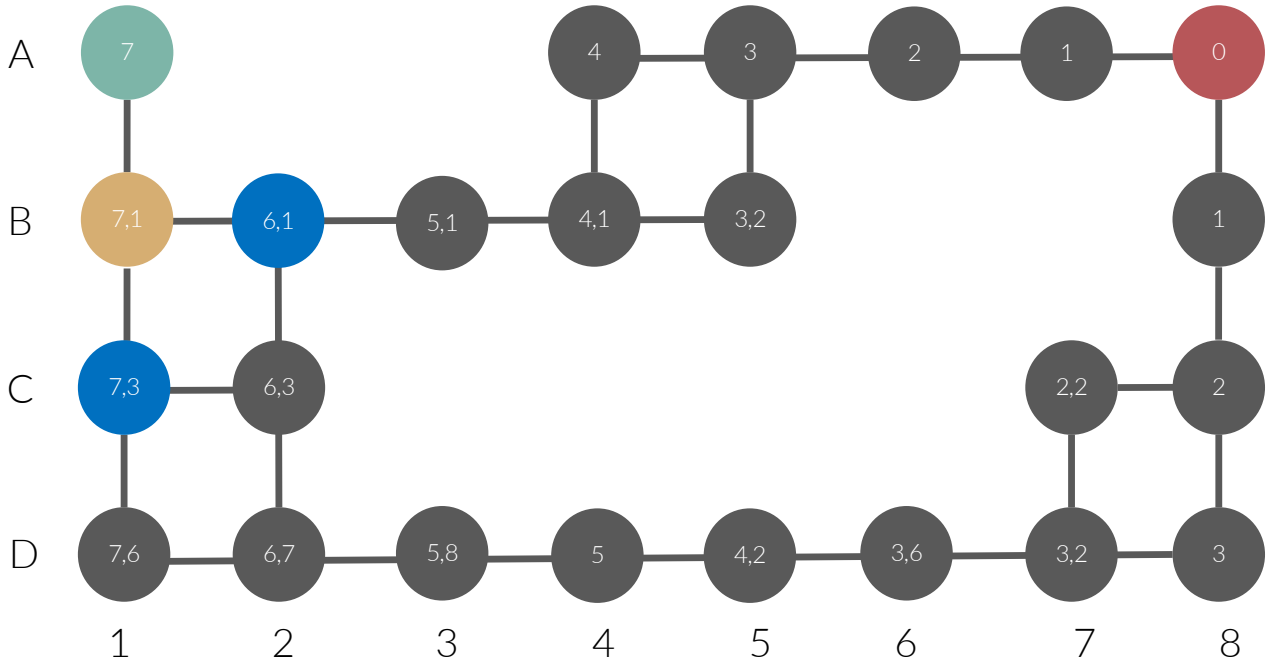
Planning as a shortest path problem: A*



Node	Value	Parent
B1	7,1+1=8,1	A1

Value (F) = heuristic cost (H) + walking cost (G)

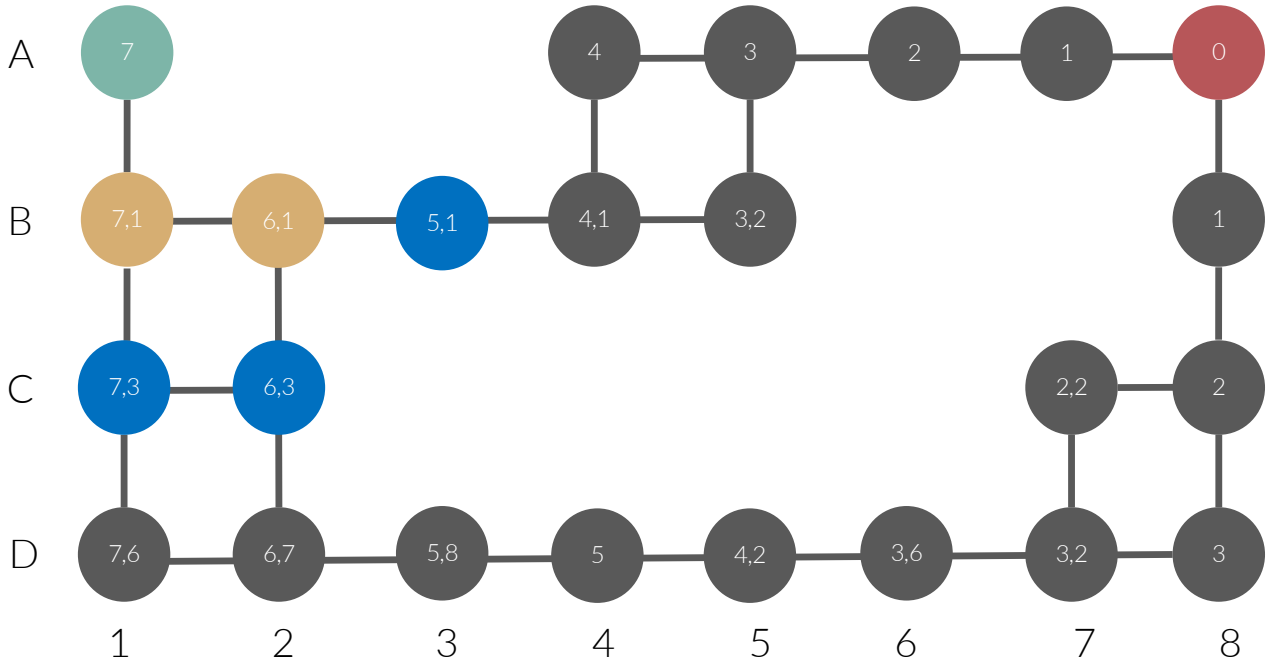
Planning as a shortest path problem: A*



Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1

To choose the next nodes: start from the node in the open list (blue nodes) with the lowest value (F)!

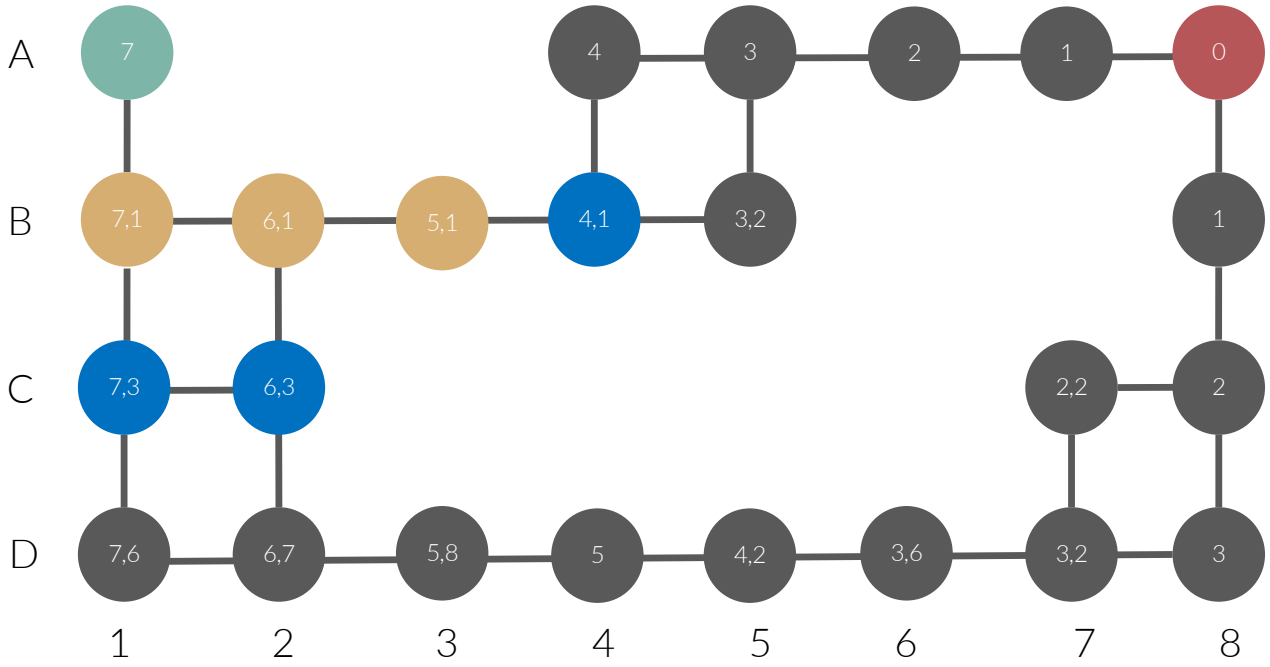
Planning as a shortest path problem: A*



Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2

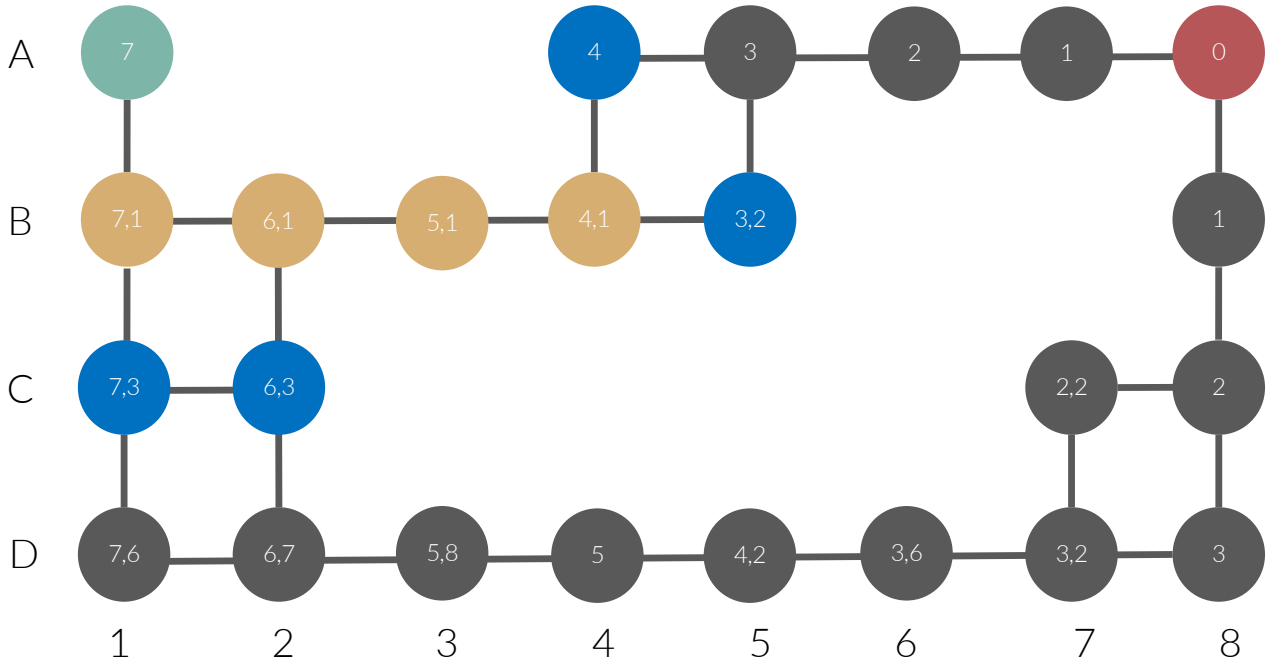
To choose the next nodes: start from the node in the open list (blue nodes) with the lowest value (F)!

Planning as a shortest path problem: A*



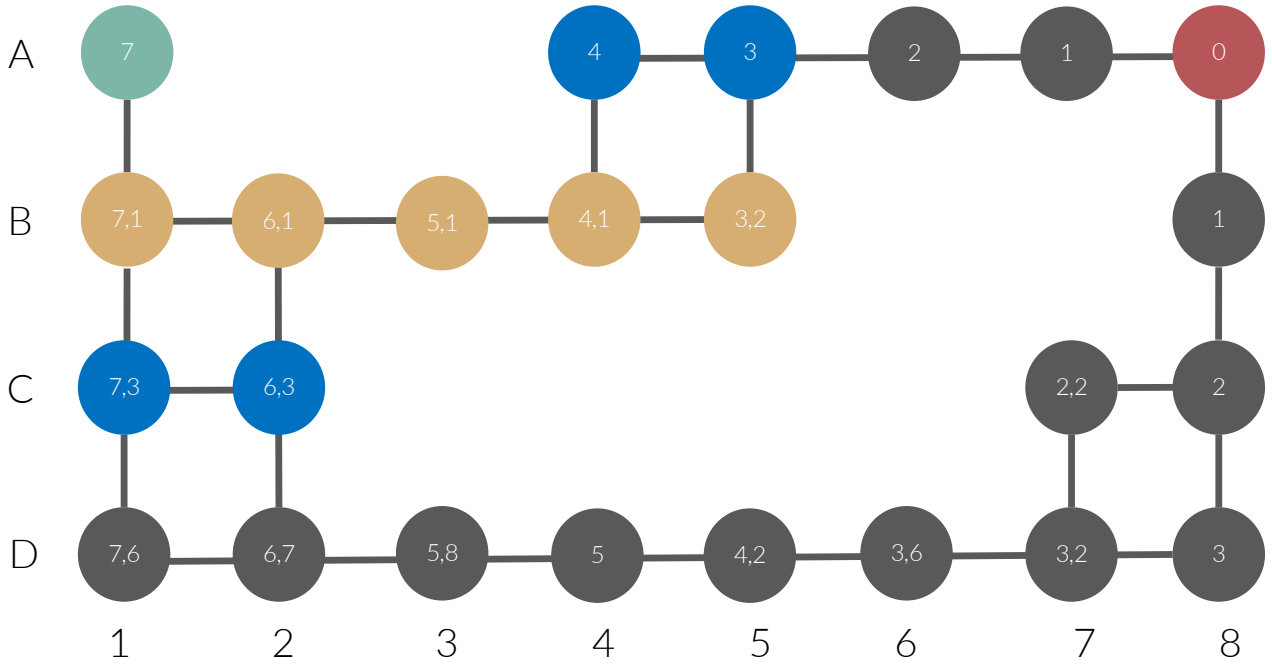
Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2
B4	4,1+4=8,1	B3

Planning as a shortest path problem: A*



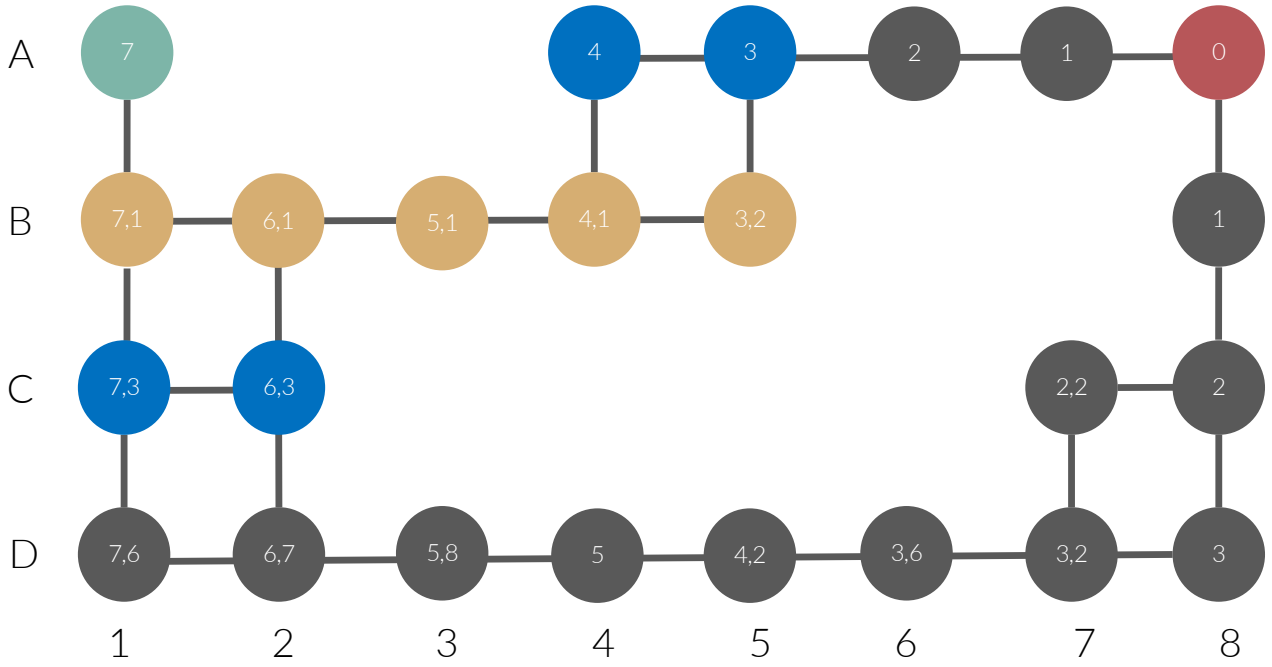
Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2
B4	4,1+4=8,1	B3
A4	4+5=9	B4
B5	3,2+5=8,2	B4

Planning as a shortest path problem: A*



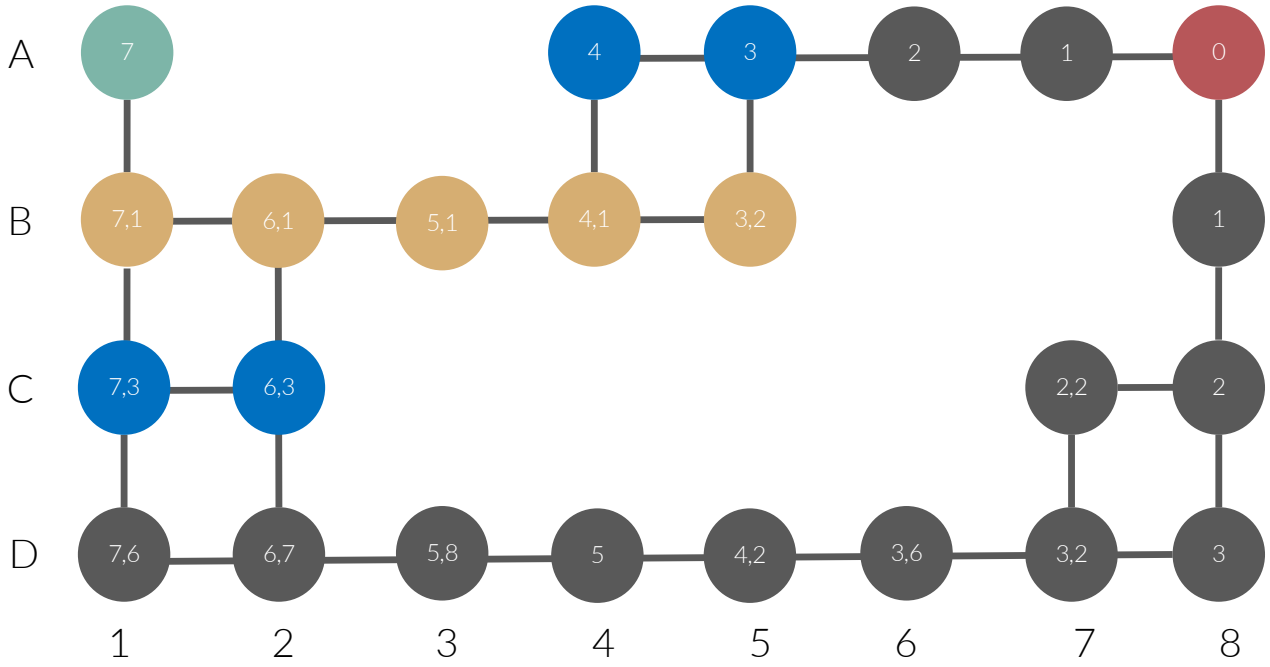
Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2
B4	4,1+4=8,1	B3
A4	4+5=9	B4
B5	3,2+5=8,2	B4
A5	3+6=9	B5

Planning as a shortest path problem: A*



Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2
B4	4,1+4=8,1	B3
A4	4+5=9	B4
B5	3,2+5=8,2	B4
A5	3+6=9	B5

Planning as a shortest path problem: A*

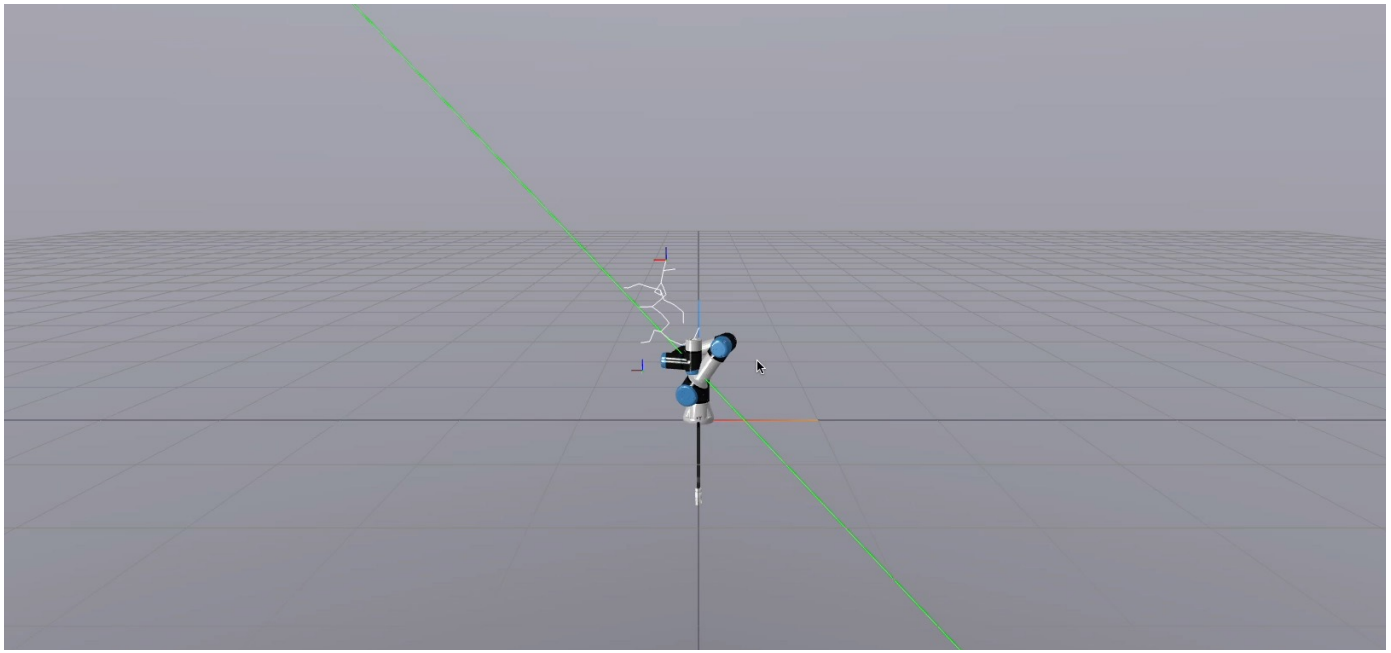


Node	Value	Parent
B1	7,1+1=8,1	A1
C1	7,3+2=9,3	B1
B2	6,1+2=8,2	B1
B3	5,1+3=8,3	B2
C2	6,3+3=9,3	B2
B4	4,1+4=8,1	B3
A4	4+5=9	B4
B5	3,2+5=8,2	B4
A5	3+6=9	B5

Etc -> complete as an exercise

Sampling-based methods

Sampling-based methods



Sample in the C-space or work space

Evaluate sample (collisions etc)

Add this sample to the nearest free-space samples

Rapidly exploring random tree (RRT)

```
1:  function RRT(q_init, q_goal, K, Δq)
2:      INIT(T, q_start)
3:      for i = 1 to K do
4:          q_rand ← RANDOM_STATE
5:          q_near ← NEAREST(T, q_rand)
6:          q_new ← NEW_CONFIG(q_near, q_rand, Δq)
7:          if COLLISION_FREE(q_near, q_new) then
8:              ADD_VERTEX(T, q_new)
9:              ADD_EDGE(T, q_near, q_new)
10:             if IS_CLOSE(q_new, q_goal, ε) then
11:                 return PATH(T, q_init, q_goal)
12:             end if
13:         end if
14:     end for
15:     return FAILURE
16: end function
```

graph

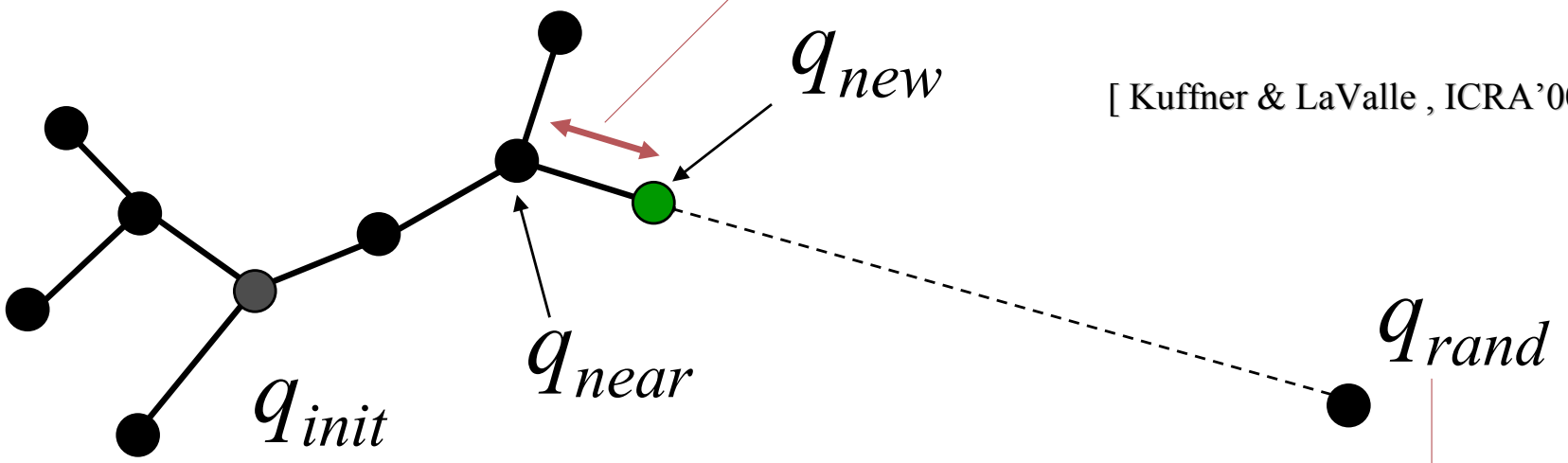
we add it to the tree

We reached our goal configuration

we didn't find a path

Note: can be performed in C-space, but also in task space

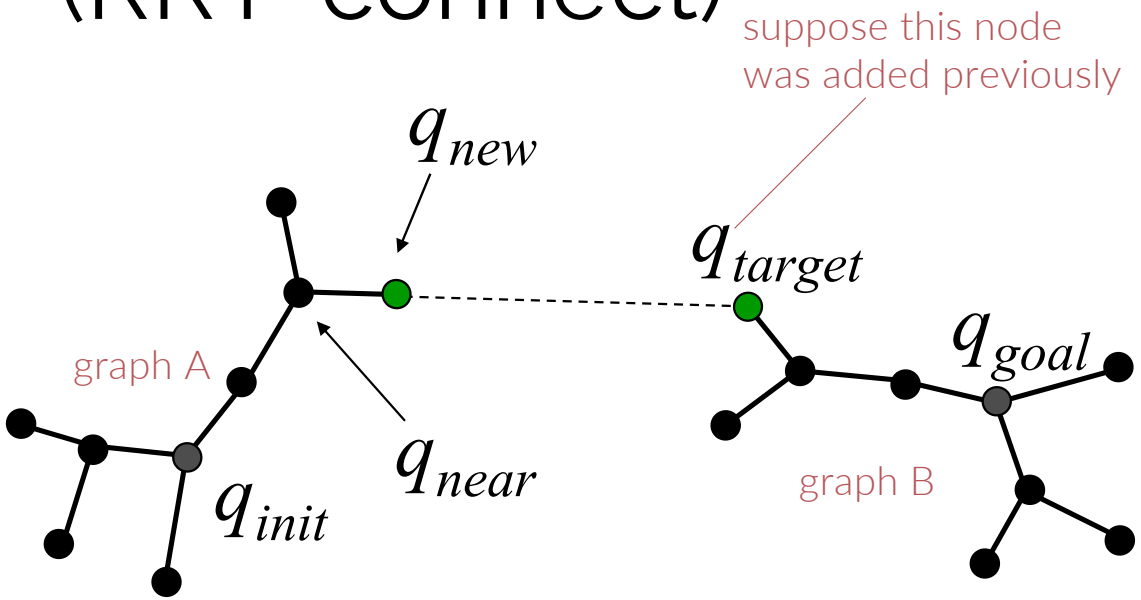
Rapidly exploring random tree (RRT)

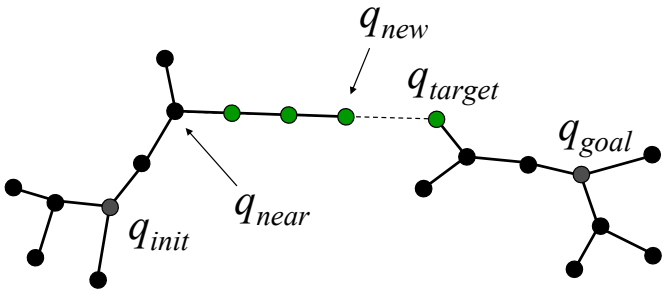


[Kuffner & LaValle , ICRA'00]

It can be interesting to sample with a low probability the goal configuration instead of a random configuration

Rapidly exploring random tree Connect: (RRT-connect)





You can try to keep on extending the branch

CONNECT($\mathcal{T}, q$)
1 **repeat**
2 $S \leftarrow$ EXTEND($\mathcal{T}, q$);
3 **until not** (S = *Advanced*)
4 Return S;

RRT_CONNECT_PLANNER(q_{init}, q_{goal})
1 $\mathcal{T}_a$.init(q_{init}); $\mathcal{T}_b$.init(q_{goal});
2 **for** $k = 1$ **to** K **do**
3 $q_{rand} \leftarrow$ RANDOM_CONFIG();
4 **if not** (EXTEND($\mathcal{T}_a, q_{rand}$) =*Trapped*) **then**
5 **if** (CONNECT($\mathcal{T}_b, q_{new}$) =*Reached*) **then**
6 Return PATH($\mathcal{T}_a, \mathcal{T}_b$);
7 SWAP($\mathcal{T}_a, \mathcal{T}_b$);
8 Return *Failure*

Luckily, many good software toolboxes exist

- Open Motion Planning Library (OMPL)
  - Collection of SOTA planners
  - We use it in the AIRO-mono Repo
  - https://ompl.kavrakilab.org
- Isaac CuRobo (NVidia)
  - Exploits the parallellisation capabilties on GPU
  - https://curobo.org

Optimisation-based

Moving your robot – Recap

- FK: joint space → task space
- IK: task space → joint space
  - Analytical: fast but not always available
  - Numerical: quick to implement but singularities
  - Optimization: slower, prone to local minima but *rich* for task expression
- Path planning: produces a path = set of ordered points (in C-space or task space)
- Motion planning: produces a trajectory = time-parametrized path

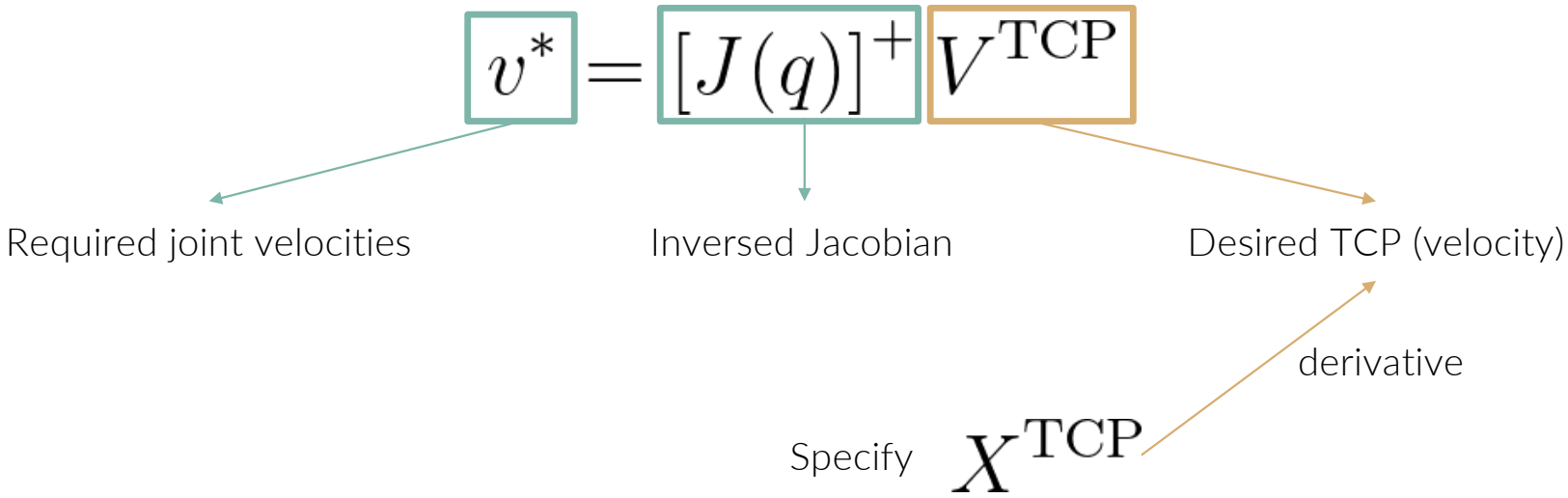
Differential IK: algorithm - recap

1. Calculate the error *e* between the actual and desired pose;
2. Compute the Jacobian;
3. Invert* the Jacobian;
4. Calculate $\Delta\theta = J^* e$
5. Move $\theta = \theta + \alpha\Delta\theta$
6. Repeat 1-5 until the error *e* gets small

Multiple options:
- transpose
- pseudo inverse
...

Differential IK as Cartesian controller

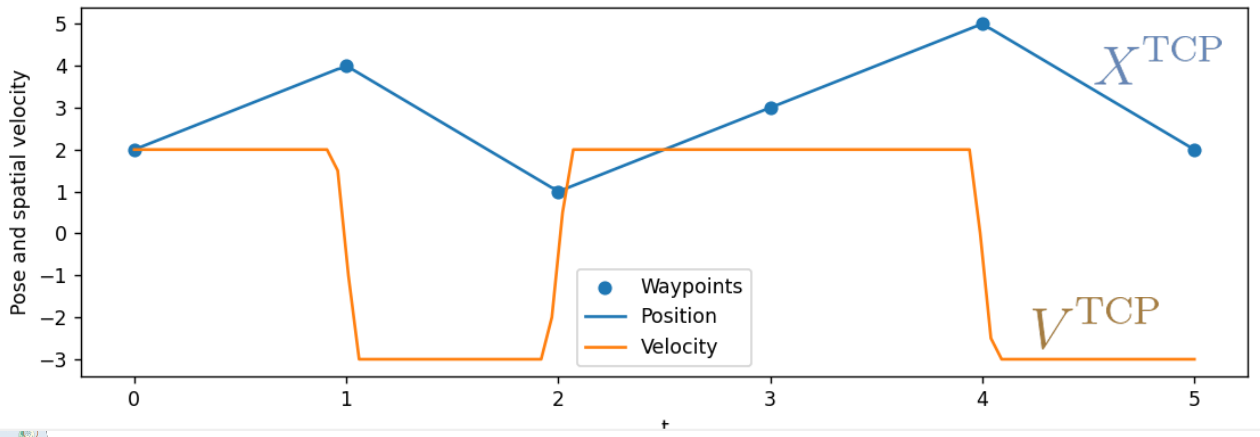
- You can produce a motion plan with the differential IK algorithm you already implemented (i.e., a pseudo-inverse Cartesian controller)



Differential IK as Cartesian controller

$$v^* = [J(q)]^+ V^{\mathrm{TCP}}$$

Desired TCP (velocity)



Differential IK as Cartesian controller

Problems

- Singularities causes large joint velocities (cfr. course 2 – kinematics)
- Very difficult to add constraints making this formulation less descriptive
    - E.g., velocity constraints, scene collisions, self-collisions, …

A word about optimization and solvers

$$\begin{aligned} \min_{x} \quad & f(x), \\ \text{subject to} \quad & \phi(x) \leq 0. \end{aligned}$$

- There is a wealth of solver packages that implement best practices:
  - Numerically conditioning the problem
  - Discard trivial constraints
  - Warm-start the solution
- There is not one dominant solver for all optimization problems
- In general, we prefer linear and convex optimization problems above nonlinear and nonconvex
  - Convex optimization has some nice properties such as having a unique global optimum and we have multiple efficient solvers for this
  - Nonlinear solvers are prone to local optima and take more time to solve
  - Our understanding of nonlinear optimization landscapes are still evolving

Differential IK: optimisation-based

$$X^{\mathrm{TCP}} = f_{\mathrm{kin}}(q) \quad \leftrightarrow \quad \textit{Definition FK}$$

$$V^{\mathrm{TCP}} = J(q)v \quad \leftrightarrow \quad \textit{First derivative}$$

$$\min_{v} \| J(q)v - V^{\mathrm{TCP}} \|^2 \quad \leftrightarrow \quad \textit{Rewrite}$$

Differential IK: optimisation-based

$$\min_{v} \| J(q)v - V^{\mathrm{TCP}} \|^2$$

From linear algebra we know that the pseudo-inverse is the optimal solution to a least-squares optimisation problem:

$$v^* = [J(q)]^+ V^{\mathrm{TCP}}$$

But once we start adding constraints (e.g., max velocities), the solution is not solely defined by the inverse Jacobian.

Optimisation-based diff IK: example

$$\min_{v} \left| J(q)v - V^{\mathrm{TCP}} \right|_2^2,$$

$$\text{subject to } v_{\min} \leq v \leq v_{\max}$$

Quadratic

Linear

Solvable with quadratic programming (QP's)

Optimisation-based diff IK: example

$$\min_{v} \left| J(q)v - V^{\mathrm{TCP}} \right|_2^2,$$

$$\text{subject to } v_{\min} \leq v \leq v_{\max}$$

$$q_{\min} \leq q + hv \leq q_{\max}$$

$$\dot{v}_{\min} \leq \frac{v - v_c}{h} \leq \dot{v}_{\max}$$

$$\text{with} \quad v_c = \text{current velocity}$$

$$h = \text{time step}$$

Linearize position and acceleration constraints

Inverse kinematics: optimization-based

- Formulation 1: Differential IK optimization-based formulation

$$\min_{v} \| J(q)v - V^{\mathrm{TCP}} \|^2$$

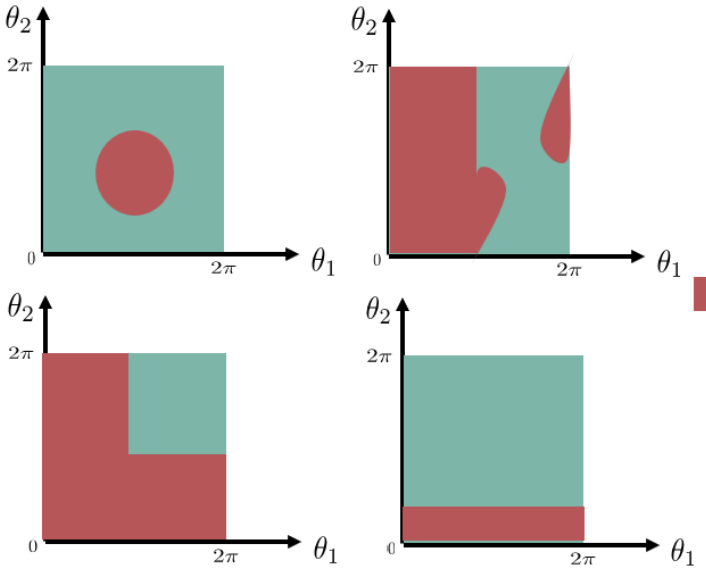
- Formulation 2: IK optimization-based formulation

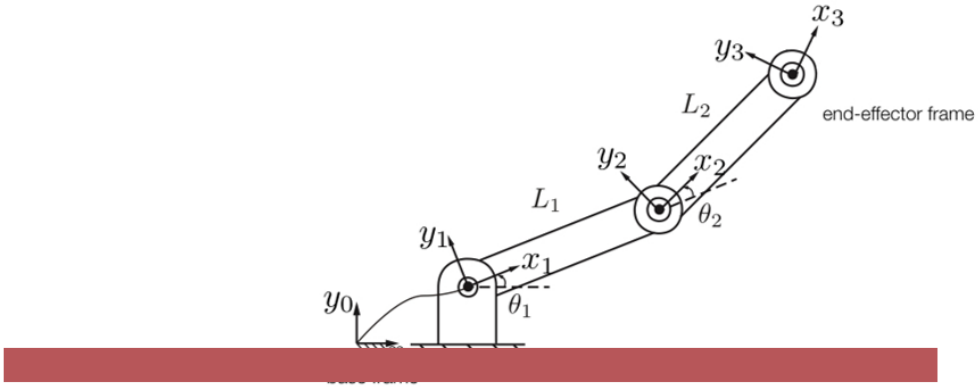
$$\begin{aligned} \min_{q} \quad & |q - q_0|^2 , \\ \text{subject to} \quad & X^{\mathrm{TCP}} = f_{kin}(q) \end{aligned}$$

Q1: Why is 2nd formulation (generally) nonlinear?

Q2: What is the difference in IK behaviour you expect between these two formulations?

Appreciation for nonlinear solvers: avoiding collisions in C-space





Optimization-based motion planning

- From inverse kinematics:

$$\begin{aligned} \min_{q} \quad & |q - q_0|^2 , \\ \text{subject to} \quad & X^{\text{TCP}} = f_{kin}(q) \end{aligned}$$

- To trajectory optimization (C-space formulation):

$$\begin{aligned} \min_{\alpha, T} \quad & T, \\ \text{subject to} \quad & X^{TCP_{\text{start}}} = f_{kin}\left(q_\alpha(t=0)\right), \\ & X^{TCP_{\text{goal}}} = f_{kin}\left(q_\alpha(t=T)\right), \\ & |\dot{q}_\alpha(t)| \leq v_{\max} \quad \forall t \end{aligned}$$

- T = total duration
- α = parametrization of the trajectory

e.g., cubic polynomial: $q_\alpha(t) = a_3 t^3 + a_2 t^2 + a_1 t + a_0$

then $\alpha = (a_0, a_1, a_2, a_3)$

Parametrization of trajectories

- $q_\alpha(t)$: trajectory parametrization is a design variable
  - Think of this as a hyperparameter
- Different options for parametrization
  - Polynomials
  - Splines
  - Neural networks
- Central idea: leverage mathematical properties to get smooth trajectories.
  - B-splines are a popular choice because they interpolate points while having a continuous 2nd derivative (i.e., acceleration)
  - Q: why do we want smooth trajectories as opposed to jerky?

Collision checking for trajectory optimization with B-splines

$$
\begin{aligned}
\min_{\alpha, T} \quad & T, \\
\text{subject to} \quad & X^{\text{TCP}_{\text{start}}} = f_{kin}\left(q_\alpha(t=0)\right), \\
& X^{\text{TCP}_{\text{goal}}} = f_{kin}\left(q_\alpha(t=T)\right), \\
& |\dot{q}_\alpha(t)| \leq v_{\max}, \\
& d(O_i(t)), R_j(t))) \geq 0 && \forall i \in [1, \ldots, N_{\text{obstacles}}], \\
& && \forall j \in [1, \ldots, N_{\text{links}}], \\
& d(R_i(t)), R_j(t))) \geq 0 && \forall i, j \in [1, \ldots, N_{\text{links}}].
\end{aligned}
$$

We typically add collision constraints at discrete timesteps. This does not guarantee the whole trajectory will be collision free!

Optimization-based planning in practice

- Choose your polynomial
- Nonlinear solvers will get stuck in local optima
  - Massage your solver!
- Good initial guesses
  - First solve without “difficult” constraints
  - Provide waypoints
  - Provide an initial trajectory
- Tune your cost function
- Keep your optimization space open by limiting the amount of constraints

Motion planning in practice

Bringing everything together

- Plan incremental movements with differential IK
- First try to plan with a naïve planner that interpolates in task space
  - Path might be jerky so post-process
- Try sample-based planners (RRT) to construct a path
  - If success, time parametrize the path by sampling waypoints and interpolate with a curve
- On failure, use failed results as warm start for trajectory optimization algorithms.

Two-stage planners

1. Construct a coarse, global path with sample-based planners
2. Use this as warm start for optimized-based planners that create a densely, interpolated trajectory

Decisions to make when planning

- Try all planners or only one?
- Waypoints (to help solver)
- Tolerances on end pose (pos + rot)
- # collision checks along path
- Trajectory parametrization type and corresponding parameters
- Tradeoff between duration and safety

No Free Lunch theorem for motion planning

- Motion planning is still an unsolved problem
- There is no canonical planner for every problem
  - Best planner is domain- and problem specific
- Find the one planner that works well for your problem
- Different planners have different features
  - Speed
  - Smoothness of movements
  - Expressiveness (adding task-specific constraints)

Next practical session is in 3 weeks, we will publish homework next week

Enjoy your holidays!